

Reinforcement Learning

Planning and Learning with Tabular Methods

Sutton/Barto* Chapter 8

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction,
2nd edition, MIT Press, Cambridge, MA, 2018



Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\mathbf{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\mathbf{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bold)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution $p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X , i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\operatorname{argmax}_a f(a)$	a value of action a at which $f(a)$ takes its maximal value

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
Q, Q_t	array estimates of action-value function q_π or q_*
$\bar{V}_t(s)$	expected approximate action value
U_t	target for estimate at time t

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r s, a)$	probability of transition to state s' and receiving reward r , from state s taking action a
	expected immediate reward from state s after action a
$\pi(a s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Planning vs. Learning

Planning vs. Learning

Model-based RL

- Requires knowledge of the MDP model.
- Methods: Dynamic Programming, heuristic search
- Use the known model to **plan**

Model-free RL

- Model is unknown.
- Methods: MC, TD
- **Learn** from interactions with the environment (sampling).

All methods:

- Compute a value function
- Look ahead to future events and compute a backed-up value to update an approximate value function.

Models and Planning

Model of the environment:

- Anything the agent can use to predict the outcome of actions (e.g., next state and reward)

Models can be:

- Distribution models: produce a distribution of all possible outcomes
- Sample models: produce (simulate) one sampled outcome for an action.

General Approach



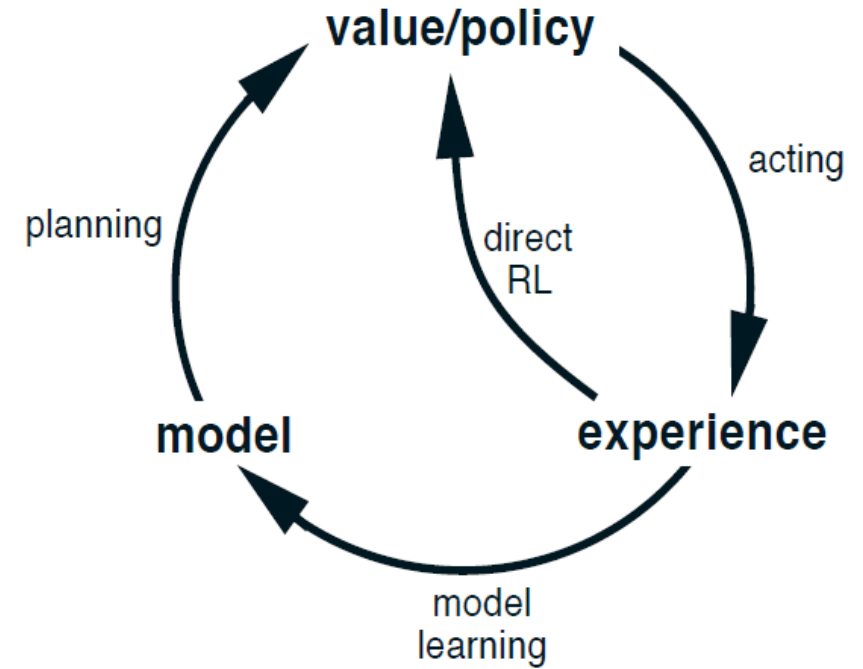
- Model-based: Simulated experience comes from the model. Can be done offline, which leads to planning.
- Model-free: Simulated experience comes from real experience, leads to online learning.

Dyna

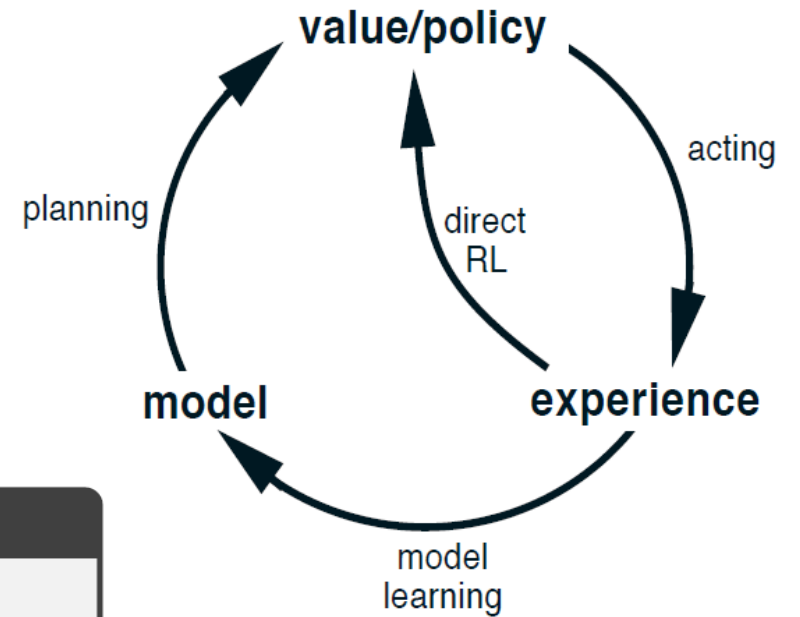
Integrating Planning, Acting, and Learning

Dyna: Integrating Planning, Acting, and Learning

- Model Learning:
 - Real experience is used to learn a model. E.g., by recording for each $S_t, A_t \rightarrow R_{t+1}, S_{t+1}$
 - The model can be used for planning to improve the value function/policy. E.g., by sampling from the model to update the Q-values.
- Direct RL:
 - Use the experience directly to improve the value function/policy. E.g., via one-step Q-learning.



Alg. Tabular Dyna-Q



Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

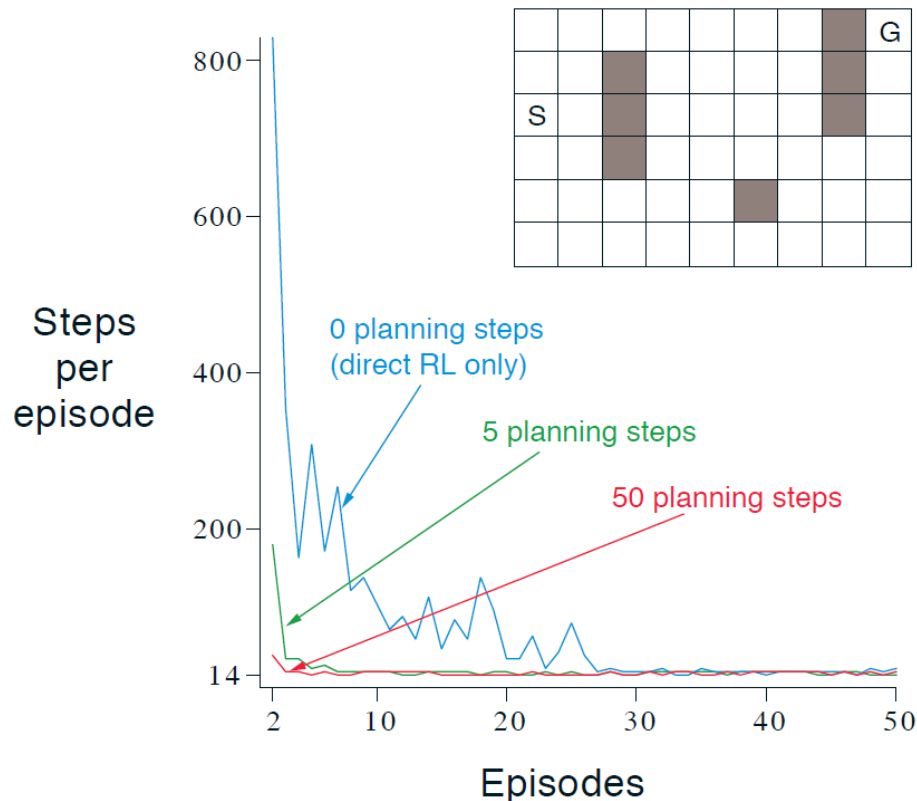
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$

Direct RL

Model
learning
and
planning

The Effect of Planning

Planning helps!



- Planning is a problem if the model is wrong:
 - Based on an insufficient number of samples
 - Function approximation not great.
 - Environment changes
- Dyna quickly corrects too optimistic predictions since they will be selected for the next real action.
- For changing environments:
 - Keep exploring.
 - Keep track of long action/state combinations that were not tried.

Sampling

Base updates on sampled experience

Expected vs. Sample Updates

- Expected updates (DP)

$$Q(s, a) \leftarrow \sum_{s', r} \hat{p}(s', r | s, a) [r + \gamma \max_{a'} Q(s', a')]$$

- Sample updates (on-step TD)

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a) - Q(s, a)]$$

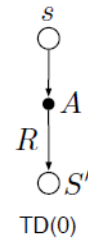
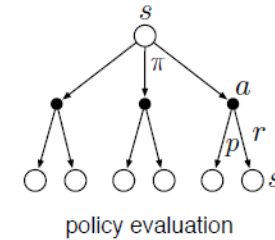
- Sampling adds sample error but is computationally much cheaper.
- Side note: If only one action is possible, they are the same for $\alpha = 1$.

Value
estimated

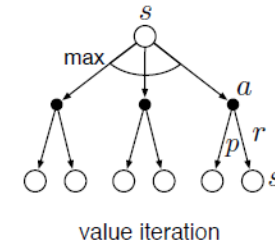
Expected updates
(DP)

Sample updates
(one-step TD)

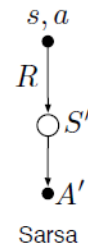
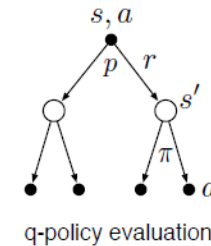
$v_{\pi}(s)$



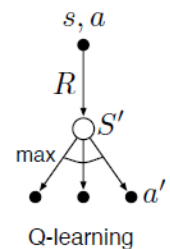
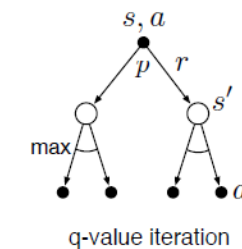
$v_*(s)$



$q_{\pi}(s, a)$



$q_*(s, a)$



Trajectory Sampling

- DP sweeps through all states in each iteration.
Converges when action/state pairs are visited ∞ times.
Issue: many states are only visited for bad policies!

Trajectory sampling

- Sample trajectories using the current policy (= uses the on-policy distribution over the states).
Converges if it continues to explore (use an ϵ -greedy policy).
- Use the samples as the experience input for any model-free method.
- Note: This is similar to off-policy MC control with an ϵ -greedy behavior policy.

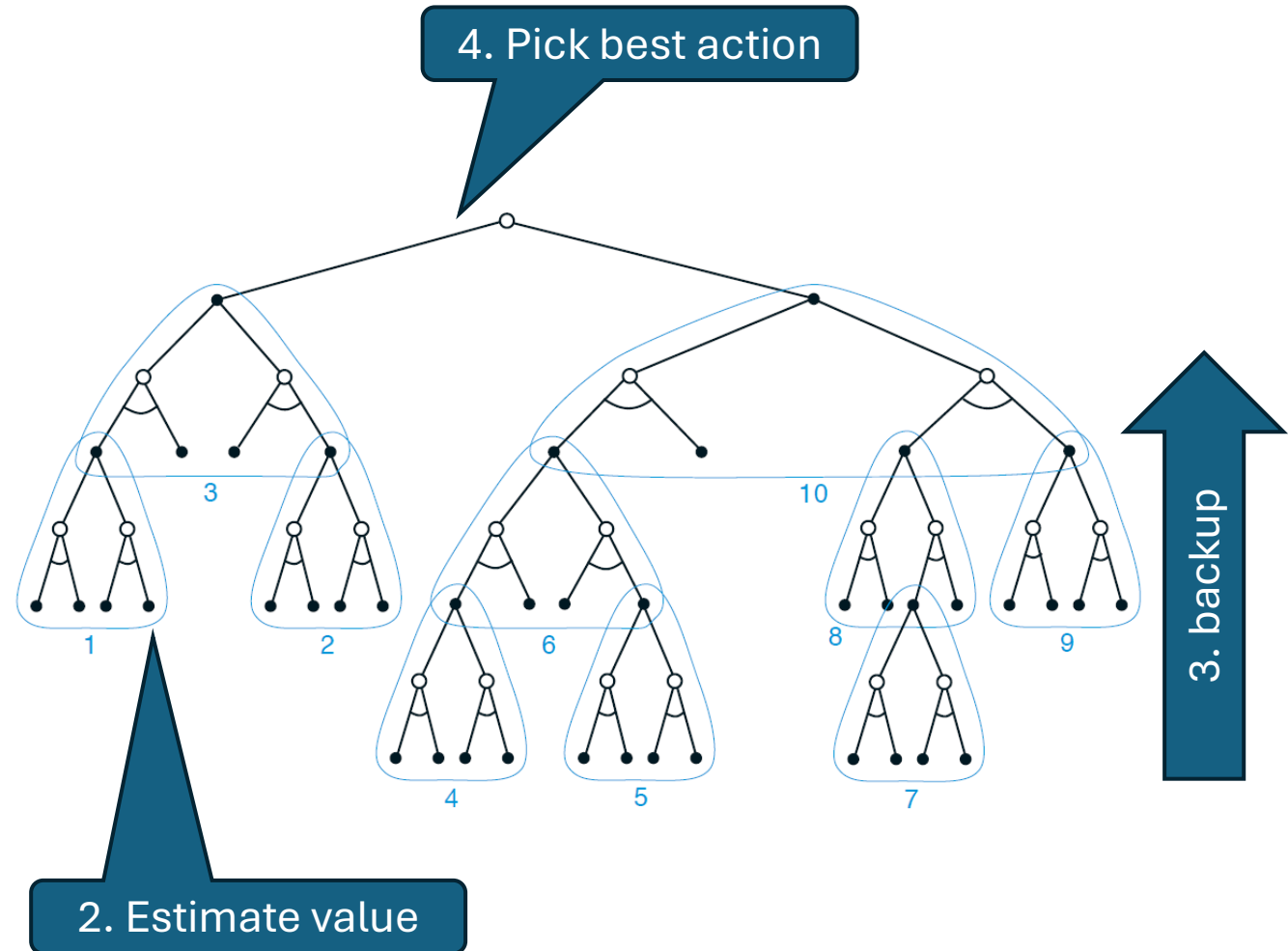
Heuristic Search and Rollout Algorithms

Decision-time planning

Heuristic Search

At decision time:

1. Consider a large tree of possible continuations (trajectories of a given length) for the current state.
2. Apply an approx. value function to the states in leaf nodes.
3. Back values towards the current state (using max).
4. Choose the best next action for the current state.

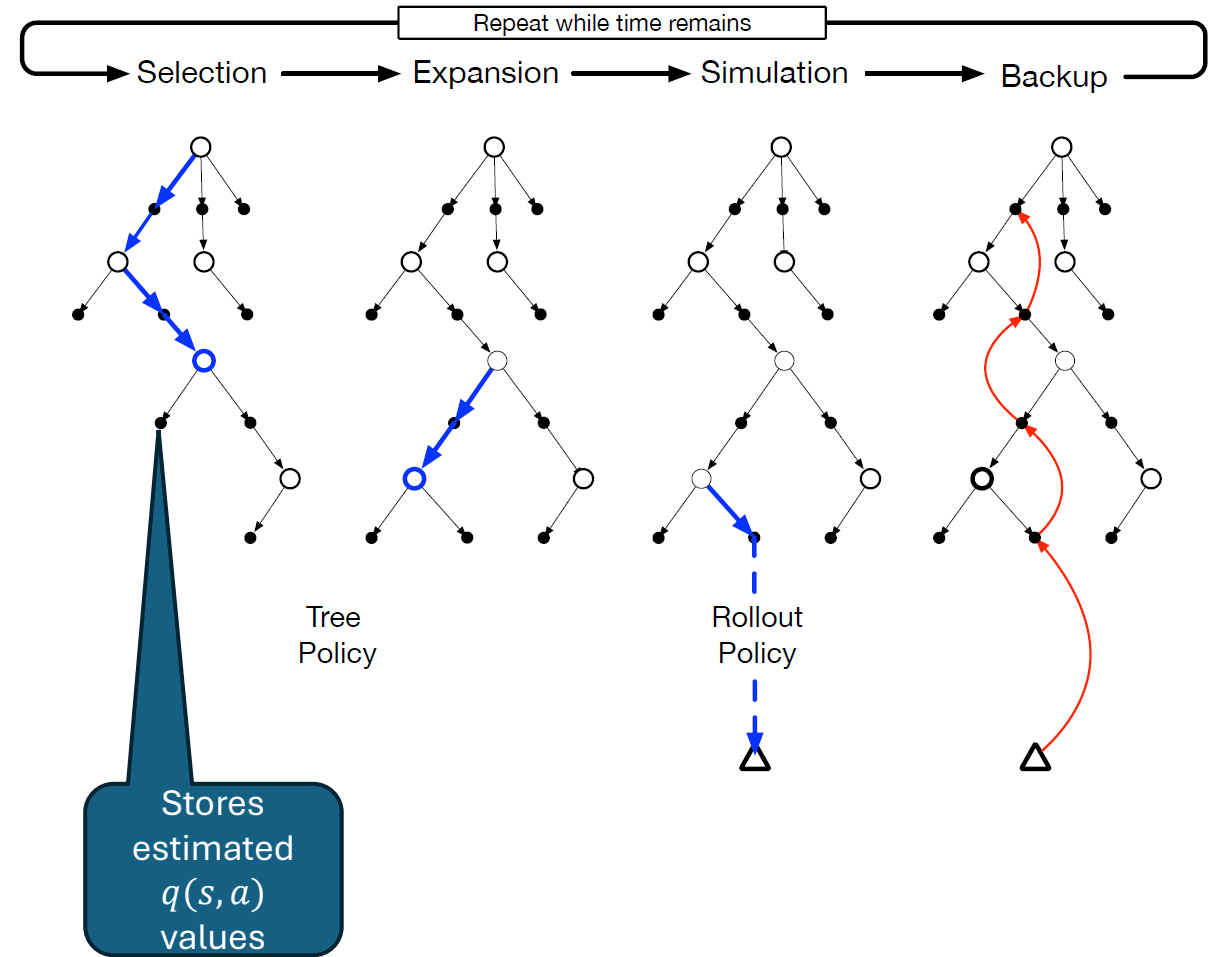


Rollout Algorithms (Monte Carlo Estimates)

- Use simulated trajectories to approximate all state/action values $q_{\pi}(s, a)$ for the current state s .
- Simulation follows a policy π called the playout policy.
- Choose the action a' greedily from the $q_{\pi}(s, a)$ estimates.
- A new policy π' that uses follows a' and then follows π is has a good chance to improve over π (policy improvement theorem).

Monte Carlo Tree Search (MCTS)

- A rollout algorithm that stores some intermediate estimates in a depth-limited tree structure.
 - Selection: Pick the currently best leaf node (e.g., greedy, UCB).
 - Expansion: Add new nodes, but keep the tree small enough.
 - Simulation: From a selected node, simulate complete episode(s).
 - Backup: update the tree with the result of the playout.
- **Advantage:** selection steers the simulation towards better trajectories. This avoids trying to approximate the complete value function.



Summary

Summary

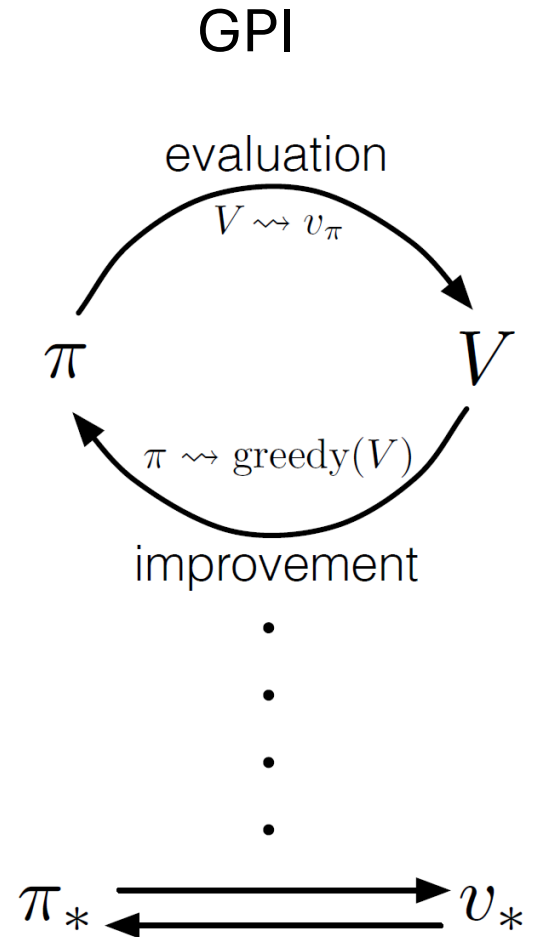
- Planning requires a model of the environment.
 - Distribution model
 - Sample model
- DP requires a distribution model because it uses expected updates.
- RL models like TD can use a sample model to perform sample updates.
- Decision-time planning can also use a sample model. E.g., rollout algorithms sample trajectories.
- Planning and learning are related: They try to estimate the same value function.

Summary of Part I

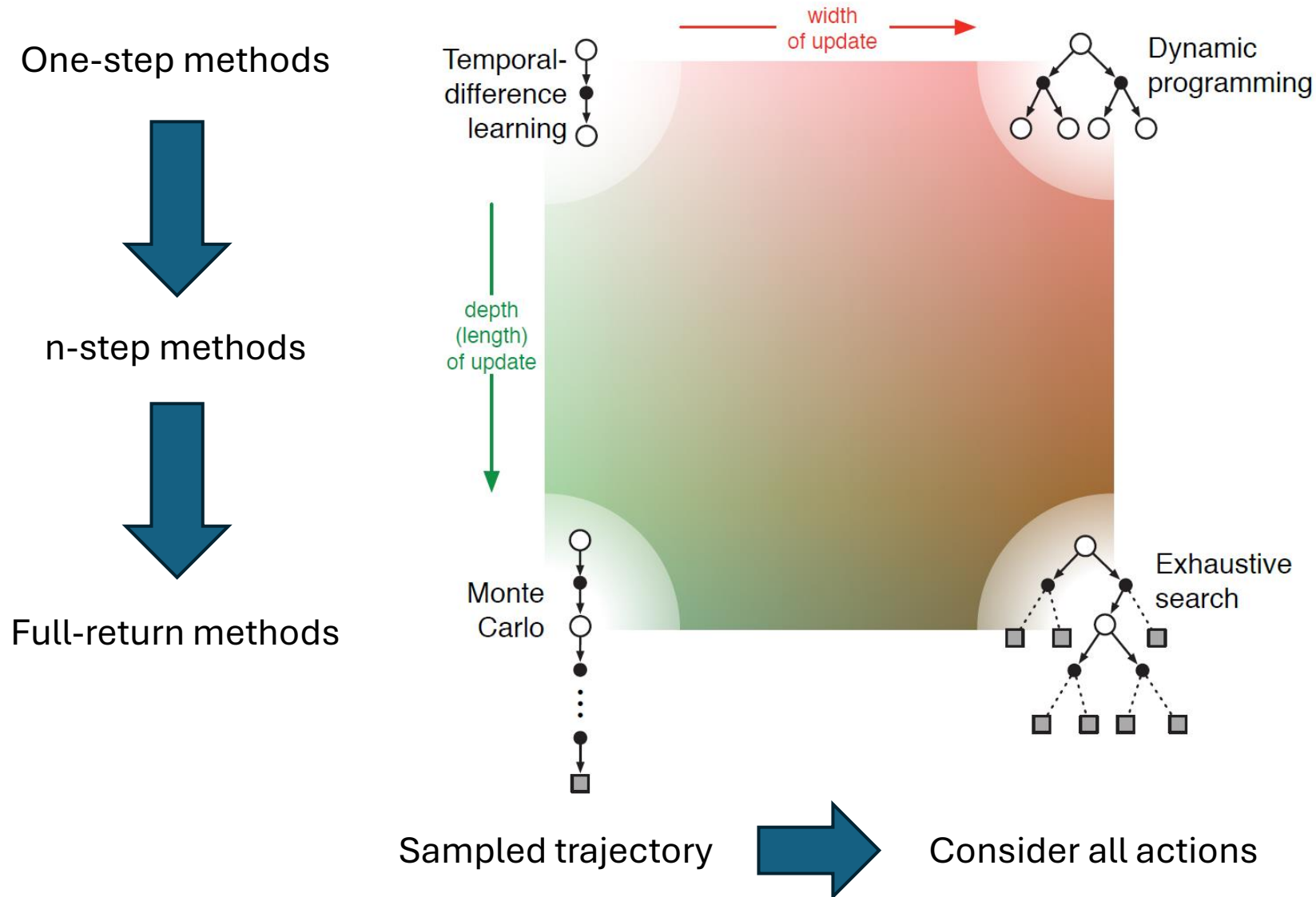
Tabular solution methods

Common Ideas

1. Estimate a value function
2. Back up along actual or possible state trajectories
3. Generalized policy iteration (GPI): maintain an approx. value function and an approx. policy. Each is used to update the other. Converges due to the policy improvement theorem.



Update Depth vs. Update Width



On-Policy vs. Off-Policy

- On-Policy: Learn the policy the agent is following
- Off-Policy: Learn a target policy that is different from the followed (behavior) policy.

Other Dimensions

- Definition of return: episodic vs. continuing task; discounted vs. undiscounted.
- Action values vs. state values
- Exploration vs Exploitation: How are actions selected to ensure a trade-off between exploration and exploitation? Many methods need exploration to converge.
- Synchronous vs. Asynchronous: Are all states updated as once or on-by-one in some order?
- Real vs. Simulated: Is real experience or simulations or both used for updates?

Issues with Tabular Methods

1. Tables need much space. A complete table with q -values has size

$$O(|\mathcal{S}| \times |\mathcal{A}|)$$

The state space size is typically extremely large or may even be infinite for continuous spaces.

2. We need a lot of data to estimate all entries.
3. Two states may be very similar and therefore should have similar q -values. Tabular methods do not take this into account.

Next, we will talk about approximate solution methods to deal with these issues.